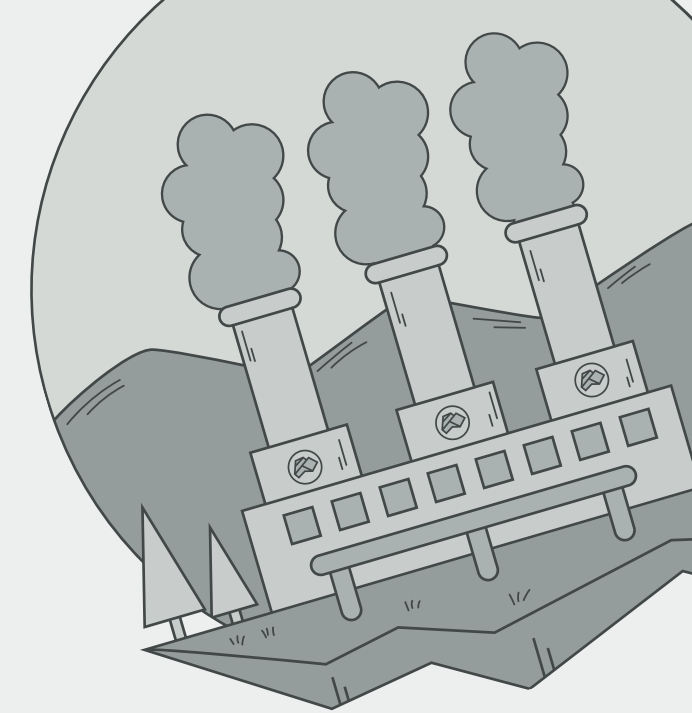
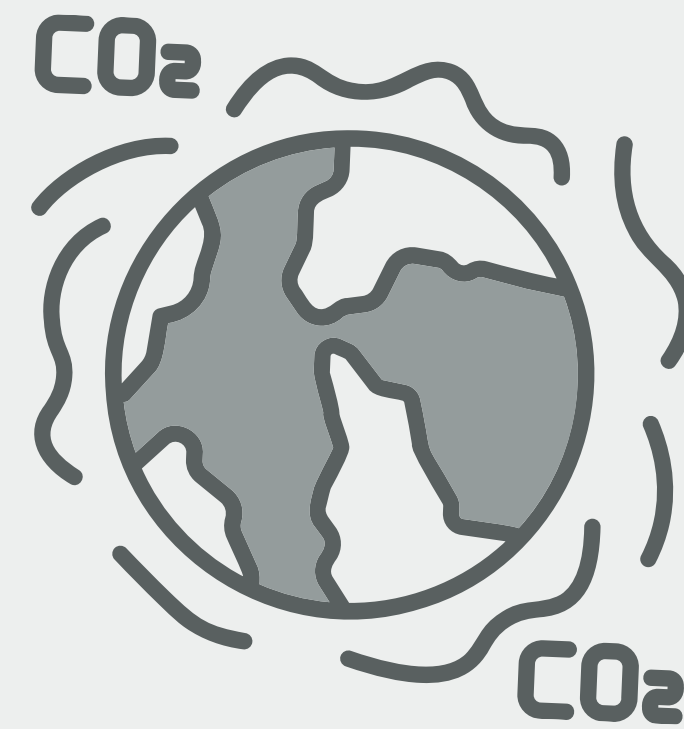
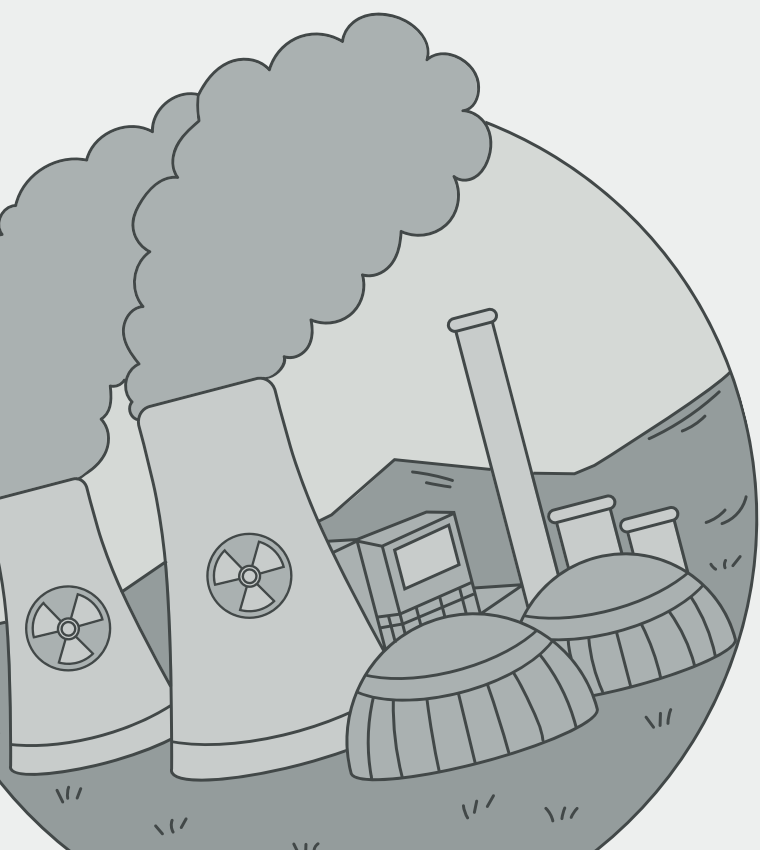




Análisis datos ambientales en Python y RStudio

Laura Juliana Nova Leaño





Introducción

En este proyecto se desarrollaron dos sistemas de análisis de datos ambientales: uno en Python y otro en R Studio.

El objetivo principal fue crear programas capaces de leer diferentes bases de datos ambientales sin necesidad de modificar el código, permitiendo trabajar con información de: temperatura, precipitación y calidad del aire.



Python

Se usaron las siguientes librerías:

```
import csv
import re
import statistics
import matplotlib.pyplot as plt
```





```
# FUNCION PARA CARGAR DATOS
```

```
def cargar_datos(nombre_archivo):
```

```
    base_datos = []  
    contador_lineas = 0
```

```
    separador = input("Ingrese el separador del archivo (; o ,): ")
```

```
    # Abrir archivo CSV
```

```
    archivo = open(nombre_archivo, 'r', encoding='utf-8')
```

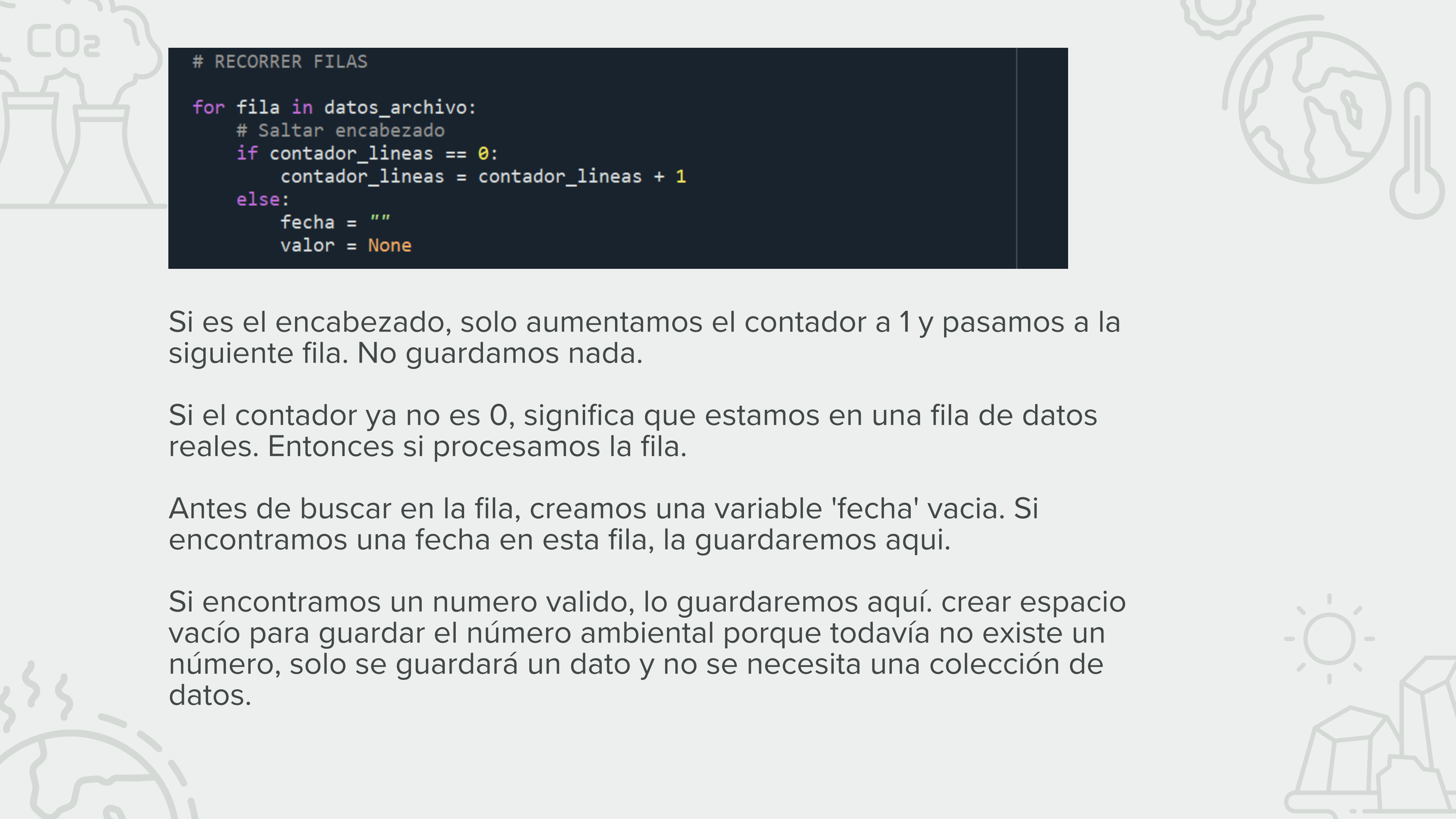
```
    # Leer archivo CSV
```

```
    datos_archivo = csv.reader(archivo, delimiter=separador)
```

Se crea una lista vacía donde se irán guardando todos los registros validos que se encuentren en el archivo.

Se crea un contador que empieza en cero. Lo usamos para saber cuándo estamos en la primera fila del archivo (el encabezado).

La 'r' significa modo lectura (read), es decir, solo vamos a leer, no a modificar. El 'encoding utf-8' permite leer correctamente tildes, símbolos, caracteres especiales.



```
# RECORRER FILAS
```

```
for fila in datos_archivo:  
    # Saltar encabezado  
    if contador_lineas == 0:  
        contador_lineas = contador_lineas + 1  
    else:  
        fecha = ""  
        valor = None
```

Si es el encabezado, solo aumentamos el contador a 1 y pasamos a la siguiente fila. No guardamos nada.

Si el contador ya no es 0, significa que estamos en una fila de datos reales. Entonces si procesamos la fila.

Antes de buscar en la fila, creamos una variable 'fecha' vacía. Si encontramos una fecha en esta fila, la guardaremos aquí.

Si encontramos un número válido, lo guardaremos aquí. crear espacio vacío para guardar el número ambiental porque todavía no existe un número, solo se guardará un dato y no se necesita una colección de datos.



```
# BUSCAR FECHA EN TODA LA FILA
```

```
for elemento in fila:
    texto = str(elemento)
    fecha_detectada = re.search(r'\d{1,4}[/-]\d{1,4}[/-]\d{2,4}', texto)

    if fecha_detectada:
        fecha = fecha_detectada.group()
```

```
# BUSCAR NUMERO EN TODA LA FILA
```

```
for elemento in fila:
    texto = str(elemento)
    texto = texto.replace(",", ".")

    # Verificar si es numero
    if texto.replace(".", "").isdigit():
        numero = float(texto)

    # Evitar años
    if numero < 500:
        valor = numero
```



Convierte el contenido de la celda en texto (string).

Usa la libreria 're' para buscar un patron de fecha dentro del texto. Esto detecta formatos como 2021-03-15, 15/03/2021.

El metodo .group() devuelve el texto exacto que coincidio con el patron, guarda la fecha encontrada en la variable fecha.

Para cada elemento de la fila, primero convierte el texto al formato numérico reemplazando la coma por punto decimal. Luego verifica que el valor sea realmente numérico antes de convertirlo a tipo float, evitando errores.





```
# GUARDAR DATOS

if fecha != "" and valor != None:
    base_datos.append([fecha, valor])
archivo.close()
return base_datos
```

Solo guardamos el registro si encontramos ambas cosas:
una fecha y un numero valido.

Esos datos se guardan usando `base_datos.append([fecha, valor])`. El comando `append()` agrega una nueva fila al final de la lista `base_datos`. Después de terminar de leer todo el archivo, `archivo.close()` cierra el CSV para liberar memoria y evitar errores, y finalmente `return base_datos` devuelve toda la base de datos organizada para poder usarla en el resto del programa.





```
# FUNCION ESTADISTICAS
def calcular_estadisticas(datos):
    if len(datos) == 0:
        return 0, 0, 0
    columna_valores = []
    for fila in datos:
        columna_valores.append(fila[1])

    media = (sum(columna_valores)/len(columna_valores))
    mediana = statistics.median(columna_valores)
    moda = statistics.mode(columna_valores)

    return media, mediana, moda
```



Antes de calcular, verifica si la lista esta vacia. 'len()' cuenta cuantos elementos tiene una lista. Si tiene 0, no hay nada que calcular y hay que evitar un error.

Si la lista esta vacia, la funcion termina inmediatamente y devuelve tres ceros

Crea una lista vacia donde se guardaran solo los numeros (sin las fechas).

De cada fila toma solo el segundo elemento (posición 1), ese segundo elemento es el valor numerico. Lo agrega a la lista 'columna_valores'.



```
# FUNCION GRAFICA
def hacer_grafica(datos):
    if len(datos) == 0:
        print("No hay datos para mostrar.")
        return
    eje_y = []
    años = []

    # EXTRAER DATOS
    for fila in datos:
        eje_y.append(fila[1])
        fecha = fila[0]

        # DIVIDIR LA FECHA
        partes_fecha = re.split(r'[/-]', fecha)

        # DETECTAR DONDE ESTA EL AÑO
        # Caso:
        # 1990-03-31
        if len(partes_fecha[0]) == 4:
            año = partes_fecha[0]
        # Caso:
        # 31/03/1990
        else:
            año = partes_fecha[-1]
        años.append(año)
```

Recorre cada registro para extraer el valor y el año.

Agrega el valor numérico de la fila a la lista del eje Y

La expresión `r'[/-]'` le indica a Python que debe separar el texto cada vez que encuentre un `/` o un `-`. Por ejemplo, si la fecha es "2020-03-15", el resultado será ['2020', '03', '15'].

`append`: agrega un valor al final de la lista.



```
# CREAR GRAFICA
plt.figure(figsize=(12,5))
plt.plot(eje_y,color='blue')
plt.title("Análisis Ambiental - Ciencias del Sistema Tierra")
plt.xlabel("Años")
plt.ylabel("Valores")

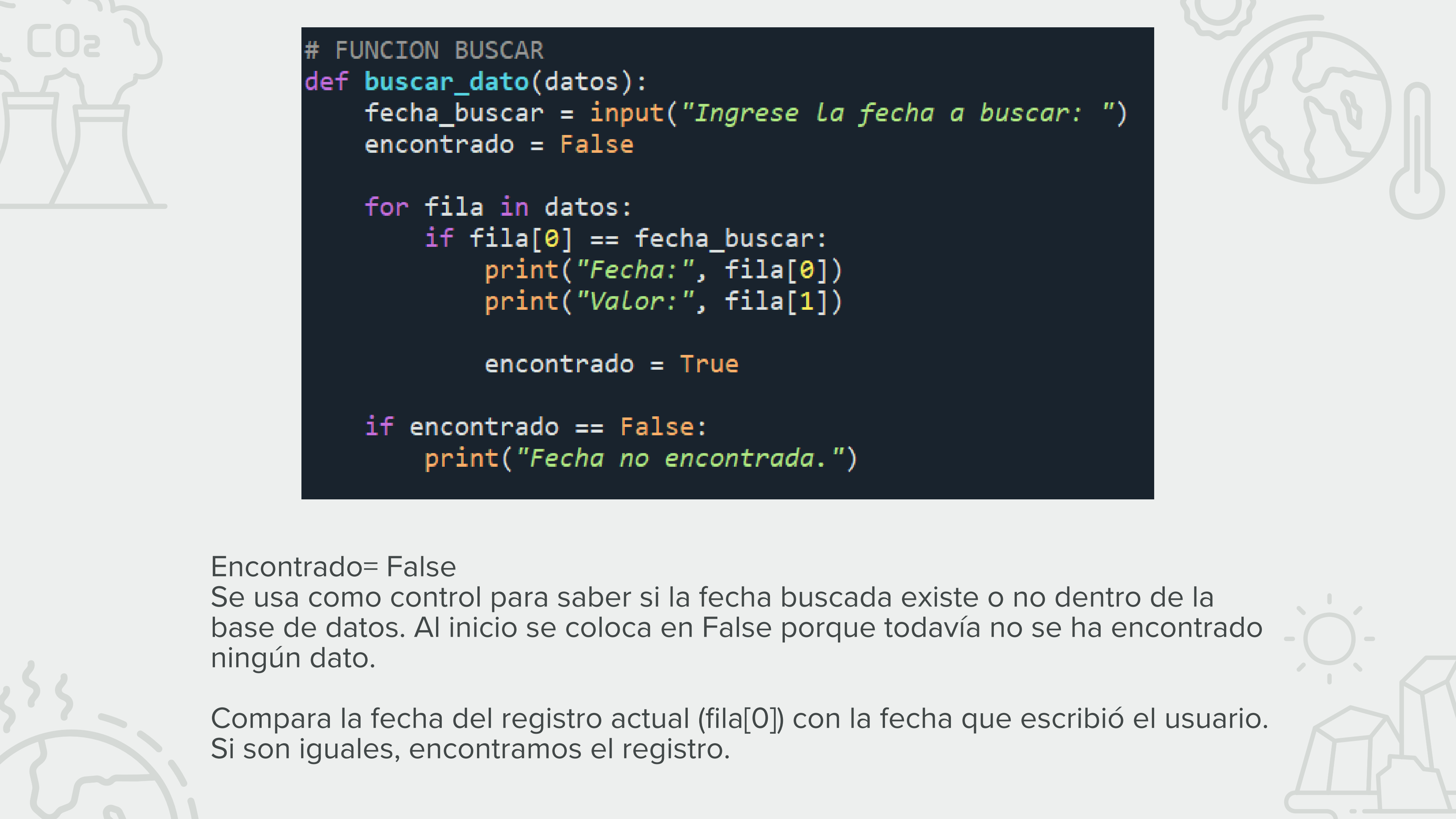
# MOSTRAR SOLO ALGUNOS AÑOS
cantidad_datos = len(años)
salto = int(cantidad_datos / 10)
if salto < 1:
    salto = 1
posiciones = []
etiquetas = []
i = 0

while i < cantidad_datos:
    posiciones.append(i)
    etiquetas.append(años[i])
    i = i + salto

plt.xticks(posiciones,etiquetas,rotation=45)
plt.grid(True)
plt.show()
```

Salto: Calcula cada cuantas posiciones mostrar una etiqueta de año.

Se verifica que el valor de salto no sea menor que 1 para evitar errores; si lo es, se ajusta a 1. Luego se crean las listas posiciones y etiquetas para almacenar los datos del eje X. Con un ciclo, el programa recorre los datos avanzando según el valor de salto, guardando las posiciones y los años correspondientes. Esto permite mostrar solo algunos años en la gráfica y evita que el eje X se vea saturado.



```
# FUNCION BUSCAR
def buscar_dato(datos):
    fecha_buscar = input("Ingrese La fecha a buscar: ")
    encontrado = False

    for fila in datos:
        if fila[0] == fecha_buscar:
            print("Fecha:", fila[0])
            print("Valor:", fila[1])

            encontrado = True

    if encontrado == False:
        print("Fecha no encontrada.")
```

Encontrado= False

Se usa como control para saber si la fecha buscada existe o no dentro de la base de datos. Al inicio se coloca en False porque todavía no se ha encontrado ningún dato.

Compara la fecha del registro actual (fila[0]) con la fecha que escribió el usuario. Si son iguales, encontramos el registro.



```
# FUNCION EDITAR

def editar_dato(datos):
    fecha_editar = input("Ingrese la fecha a editar: ")

    encontrado = False

    for fila in datos:
        if fila[0] == fecha_editar:
            nuevo_valor = float(input("Ingrese el nuevo valor: "))
            fila[1] = nuevo_valor
            print("Dato actualizado.")

            encontrado = True

    if encontrado == False:
        print("No se encontro la fecha.")
```

El float() convierte el texto que escribe el usuario en un numero decimal para poder guardarlo correctamente.





```
# FUNCION ELIMINAR
def eliminar_dato(datos):
    fecha_eliminar = input("Ingrese la fecha a eliminar: ")
    nueva_lista = []

    encontrado = False

    for fila in datos:
        if fila[0] != fecha_eliminar:
            nueva_lista.append(fila)

        else:
            encontrado = True

    if encontrado == True:
        print("Registro eliminado.")

    else:
        print("No se encontro la fecha.")

    return nueva_lista
```

Crea una lista vacía. Aquí se copiarán todos los registros excepto el que se quiere eliminar. Al final esta lista reemplazara a la original.

El programa guarda únicamente las filas que no coinciden con la fecha que se quiere eliminar



```
# MENU PRINCIPAL

def programa():
    archivo_usuario = input("Ingrese el nombre del archivo CSV (ej: Precipitacion.csv): ")

    # Cargar datos
    base_datos = cargar_datos(archivo_usuario)
    print("\nDatos cargados:", len(base_datos))
    opcion = "0"

    # MENU
    while opcion != "6":
        print("\n=====")
        print("REGISTROS:", len(base_datos))

        print("=====")
        print("1. Ver estadísticas")
        print("2. Ver grafica")
        print("3. Buscar dato")
        print("4. Editar dato")
        print("5. Eliminar dato")
        print("6. Salir")

        opcion = input("Seleccione una opcion: ")
```

len(base_datos): Muestra cuantos registros validos se encontraron en el archivo. Esto le da al usuario una confirmación de que el archivo se leyó correctamente.



```
# ESTADISTICAS

if opcion == "1":
    media, mediana, moda = (calcular_estadisticas(base_datos))
    print("\n--- RESULTADOS ---")
    print("Media:",round(media,2))
    print("Mediana:",round(mediana,2))
    print("Moda:",moda)

# GRAFICA

elif opcion == "2":
    hacer_grafica(base_datos)

# BUSCAR

elif opcion == "3":
    buscar_dato(base_datos)

# EDITAR

elif opcion == "4":
    editar_dato(base_datos)

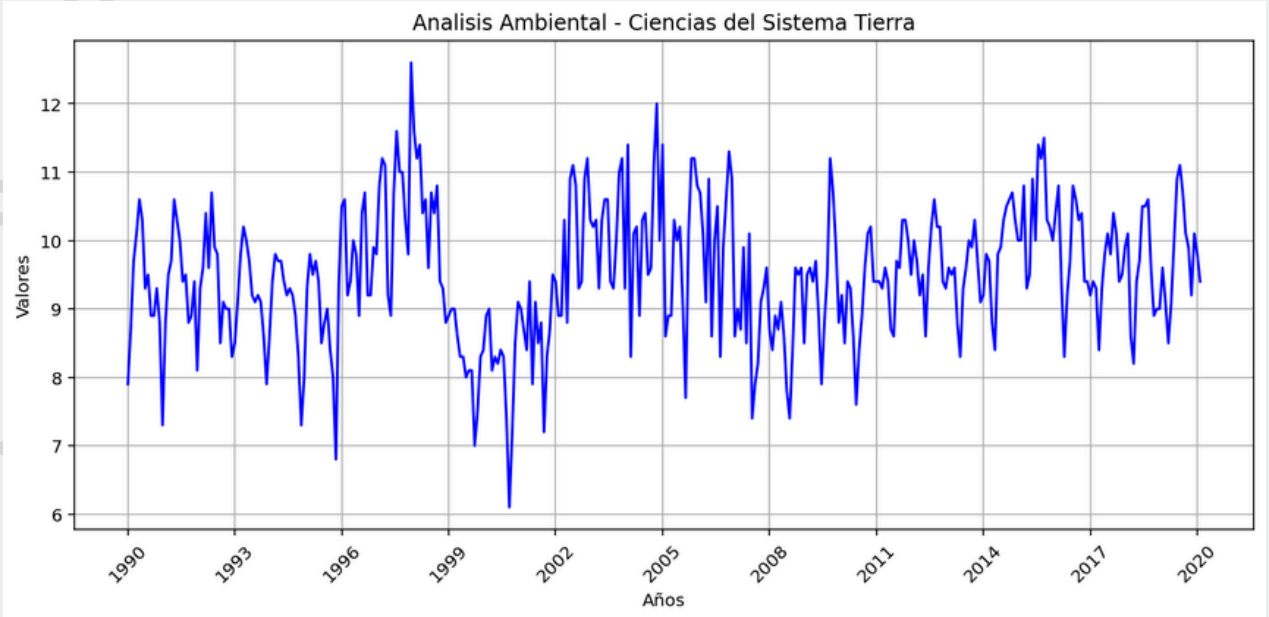
# ELIMINAR
elif opcion == "5":
    base_datos = eliminar_dato(base_datos)

# SALIR

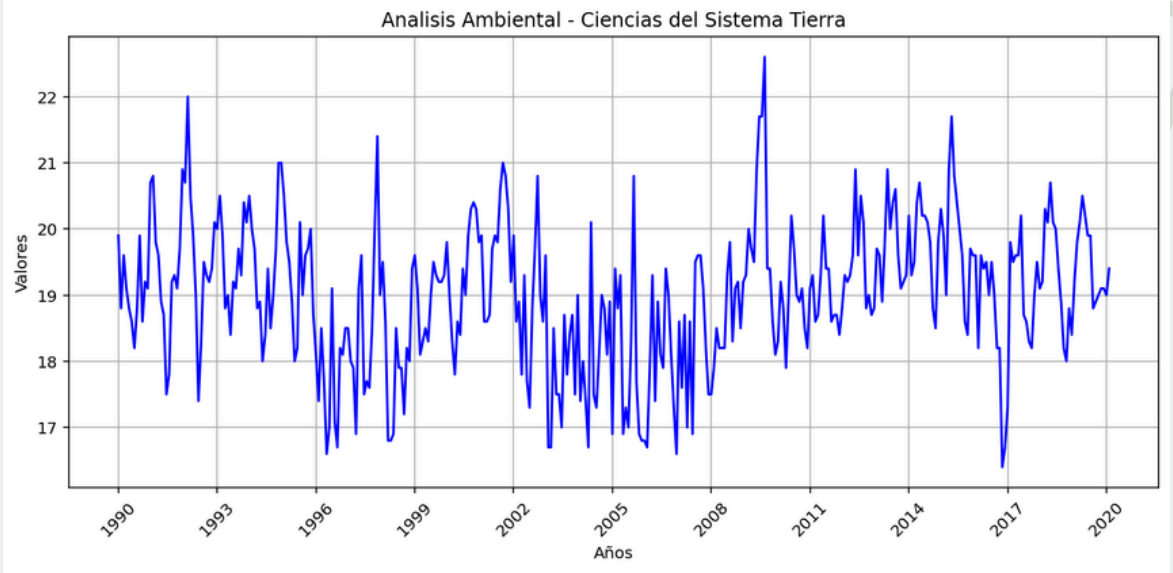
elif opcion == "6":
    print("Cerrando sistema.x")

# INICIAR PROGRAMA
programa()
```

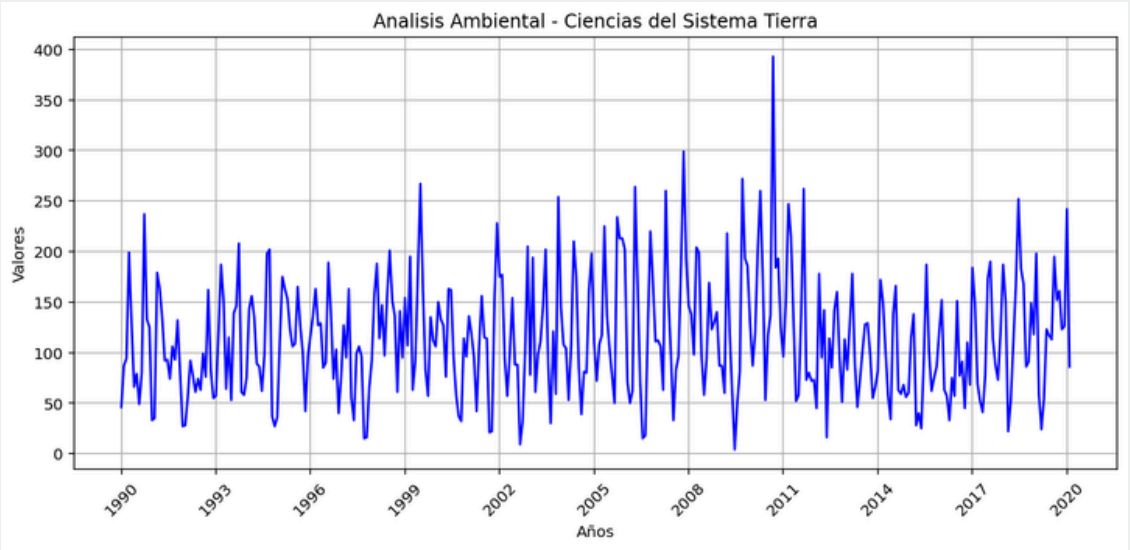




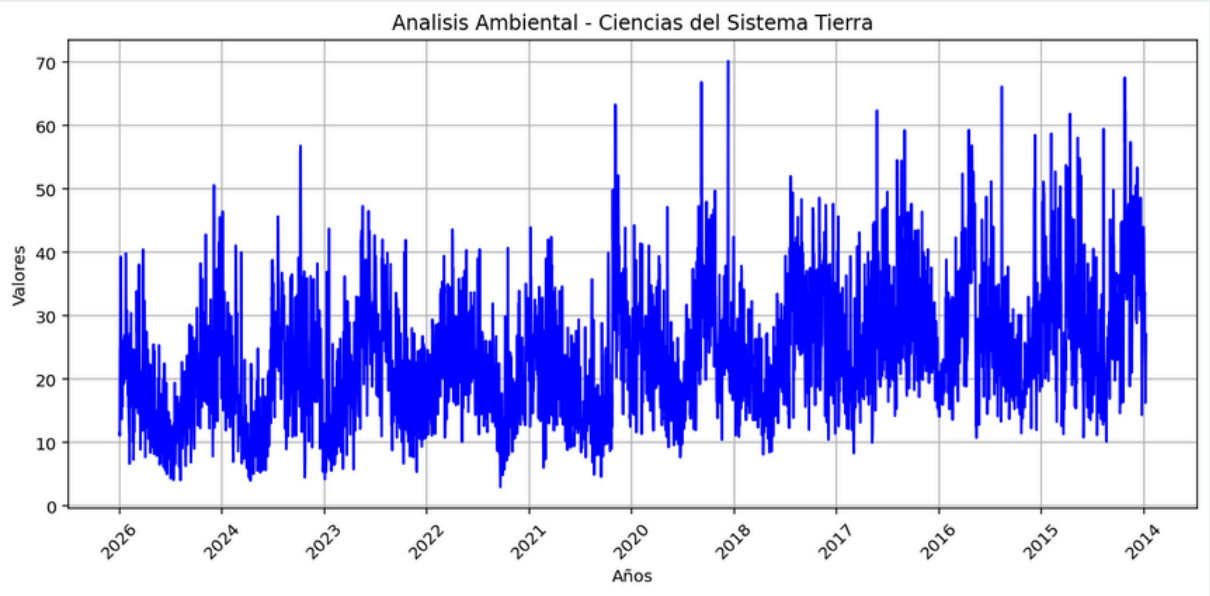
--- RESULTADOS ---
Media: 9.5
Mediana: 9.45
Moda: 9.4



--- RESULTADOS ---
Media: 19.0
Mediana: 19.0
Moda: 19.0



--- RESULTADOS ---
Media: 114.27
Mediana: 106.0
Moda: 106.0



--- RESULTADOS ---
Media: 23.86
Mediana: 22.73
Moda: 19.58333333



RStudio

Se uso la siguiente libreria:

```
library(ggplot2)
```



```
# FUNCION PARA CARGAR DATOS
```

```
cargar_datos <- function(nombre_archivo) {
```

```
  # Lista vacia donde se guardaran los datos
  base_datos <- data.frame(
    Fecha = character(),
    Valor = numeric(),
    stringsAsFactors = FALSE)
```

```
  # EL USUARIO ESCRIBE EL SEPARADOR
```

```
  separador <- readline("Ingrese el separador del archivo (; o ,): ")
```

```
  # LEER ARCHIVO CSV
```

```
  datos <- read.csv(
    nombre_archivo,
    sep = separador,
    stringsAsFactors = FALSE,
    fileEncoding = "UTF-8"
  )
```

Crea un data frame vacío con dos columnas.

stringsAsFactors=FALSE:
Evita que R convierta automáticamente los textos en categorías (factors), permitiendo trabajar las fechas como texto normal y evitando errores al buscar, comparar o editar datos



```
# RECORRER FILAS
for(i in 1:nrow(datos)) {

  fecha <- ""
  valor <- NA

  # RECORRER COLUMNAS
  for(j in 1:ncol(datos)) {
    elemento <- as.character(
      datos[i,j])
  }
}
```

Recorre cada fila del archivo CSV. `nrow()` devuelve el número total de filas. Recorre desde la fila 1 hasta la última fila de la tabla

`as.character`: Convierte cada celda a texto para poder analizar su contenido con expresiones regulares, lo que permite detectar patrones como fechas o verificar si un dato puede convertirse en número.





```
# BUSCAR FECHA CON REGEX
patron_fecha <- gregexpr("\\d{1,4}[/-]\\d{1,4}[/-]\\d{2,4}", elemento)

resultado <- regmatches(elemento, patron_fecha)

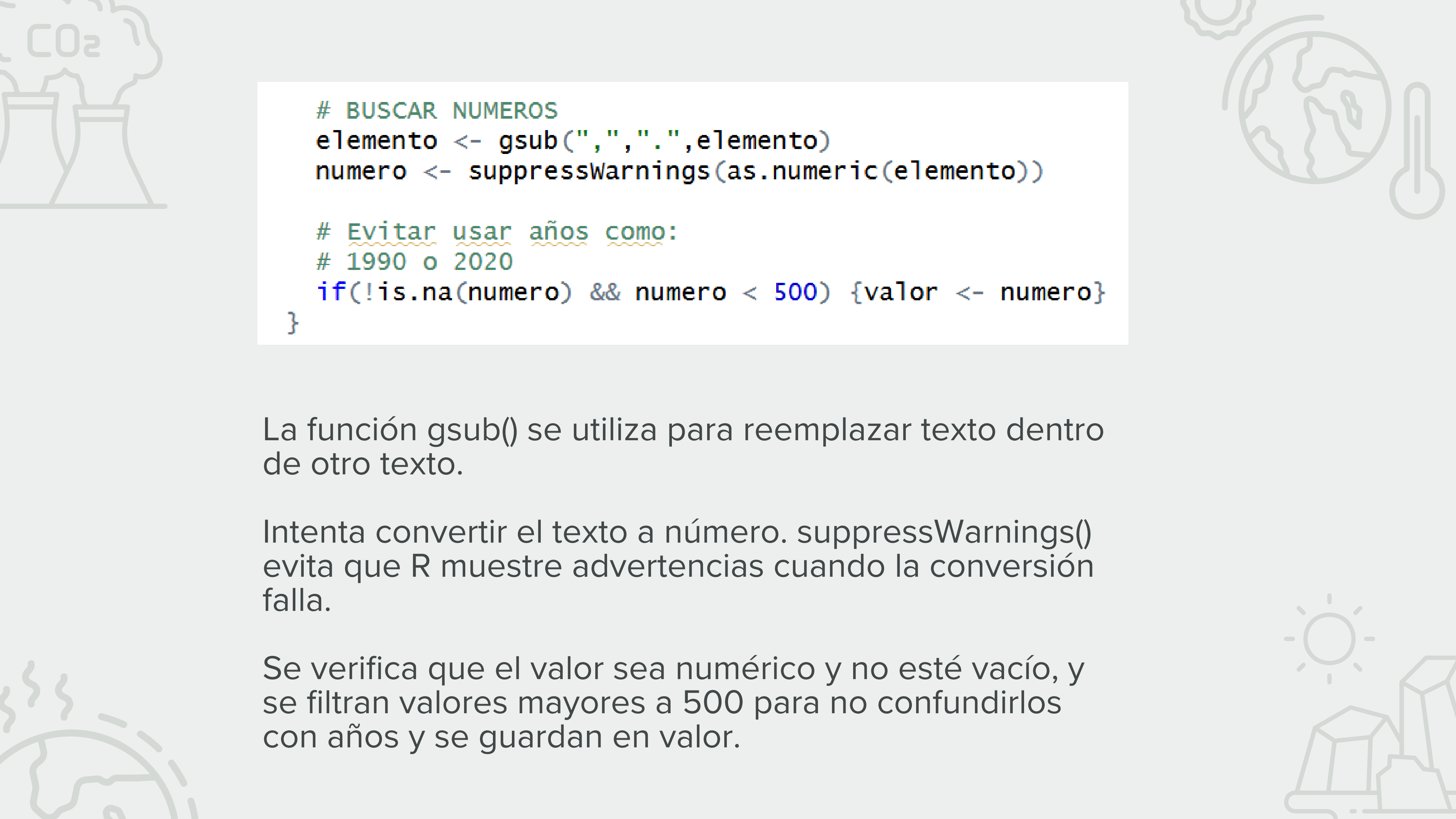
if(length(resultado[[1]]) > 0) {fecha <- resultado[[1]][1]}
```

gregexpr() sirve para encontrar todas las coincidencias de un patrón en un texto y devolver sus posiciones exactas.

La función regmatches() se utiliza para extraer el texto que coincide con un patrón encontrado anteriormente con gregexpr().

Si se encontró al menos una fecha en la celda, la guarda en la variable fecha.





```
# BUSCAR NUMEROS
elemento <- gsub(",", ".", elemento)
numero <- suppressWarnings(as.numeric(elemento))

# Evitar usar años como:
# 1990 o 2020
if(!is.na(numero) && numero < 500) {valor <- numero}
}
```

La función `gsub()` se utiliza para reemplazar texto dentro de otro texto.

Intenta convertir el texto a número. `suppressWarnings()` evita que R muestre advertencias cuando la conversión falla.

Se verifica que el valor sea numérico y no esté vacío, y se filtran valores mayores a 500 para no confundirlos con años y se guardan en `valor`.



```
# GUARDAR DATOS VALIDOS
if(fecha != "" && !is.na(valor)) {nueva_fila <- data.frame(
  Fecha = fecha,
  Valor = valor,
  stringsAsFactors = FALSE)

  base_datos <- rbind(base_datos,nueva_fila)
}

return(base_datos)
}
```



Solo guarda el registro si se encontraron ambos datos: una fecha válida y un valor numérico. Descarta filas incompletas.

La función rbind() se utiliza para agregar nuevas filas a una tabla o data.frame.





```
# FUNCION ESTADISTICAS
calcular_estadisticas <- function(datos) {

  if(nrow(datos) == 0) {

    return(c(0,0,0))}

  # MEDIA
  media <- mean(datos$Valor)

  # MEDIANA
  mediana <- median(datos$Valor)

  # MODA
  tabla <- table(datos$Valor)

  moda <- names(tabla[tabla == max(tabla)])[1]

  return(c(media, mediana, moda))
}
```

if(nrow(datos)==0
Verifica que haya datos
antes de calcular. Si el data
frame está vacío, devuelve
tres ceros para evitar
errores.

```
# FUNCION GRAFICA
hacer_grafica <- function(datos) {
  if(nrow(datos) == 0) {
    print("No hay datos para graficar")
    return()
  }

  # EXTRAER AÑOS
  años <- c()
  for(i in 1:nrow(datos)) {
    fecha <- as.character(datos$Fecha[i])

    partes <- unlist(strsplit(fecha,"/|-"))

    # Detectar el año
    if(nchar(partes[1]) == 4) {año <- partes[1]}
    else {año <- partes[length(partes)]}
    años <- c(años, año)
  }
}
```

Crea un vector vacío donde se irán guardando los años extraídos.

Dentro del ciclo, se toma el valor de la columna Fecha en la fila actual y se convierte a texto para poder manipularlo. Luego, la fecha se divide usando “/” o “-” como separadores, lo que permite extraer posteriormente el año de cada registro.

unlist() = quita la estructura de lista y deja solo los valores.

nchar() cuenta la cantidad de caracteres del primer elemento, el programa asume que el año está al inicio de la fecha.

partes[length(partes)] para tomar el último elemento de la lista.

```
# MOSTRAR SOLO ALGUNOS AÑOS
cantidad <- length(años)
salto <- round(cantidad / 10)
if(salto < 1) {salto <- 1}

posiciones <- seq(1,cantidad,by = salto)
etiquetas <- años[posiciones]

# GRAFICA
grafica <- ggplot(datos,
  aes(x = 1:nrow(datos),y = Valor)) +
  geom_line(color = "blue") +
  ggtitle("Análisis Ambiental - Ciencias del Sistema Tierra") +
  xlab("Años") +
  ylab("Valores Ambientales") +
  scale_x_continuous(breaks = posiciones,labels = etiquetas) +
  theme_minimal()
print(grafica)
}
```

Garantiza que el salto nunca sea cero (lo que causaría un bucle infinito).

La función seq(): sequence
crear una secuencia de números

Crea una lista de posiciones
comenzando en 1 y avanzando según
el valor de salto.

breaks = posiciones: indica en qué
posiciones del eje horizontal
aparecerán las etiquetas, mientras que
labels = etiquetas reemplaza esos
valores numéricos por los años
correspondientes.



```
# FUNCION BUSCAR DATO
buscar_dato <- function(datos) {
  fecha_buscar <- readline("Ingrese fecha a buscar: ")
  encontrado <- FALSE
  for(i in 1:nrow(datos)) {
    if(datos$Fecha[i] == fecha_buscar) {
      print(datos[i,])
      encontrado <- TRUE
    }
  }

  if(encontrado == FALSE) {
    print("Fecha no encontrada")
  }
}
```



Iniciada en falso, se volverá true si se encuentra al menos un resultado.

Compara la fecha del registro i con la fecha que escribió el usuario.

Imprime toda la fila i del data frame si hay coincidencia. La coma dentro de [] indica 'todas las columnas'.



```
# FUNCION EDITAR DATO
editar_dato <- function(datos) {
  fecha_editar <- readline("Fecha a editar: ")
  encontrado <- FALSE
  for(i in 1:nrow(datos)) {
    if(datos$Fecha[i] == fecha_editar) {
      nuevo <- as.numeric(readline("Nuevo valor: "))

      datos$Valor[i] <- nuevo
      encontrado <- TRUE
      print("Dato actualizado")
    }
  }

  if(encontrado == FALSE) {
    print("Fecha no encontrada")
  }

  return(datos)
}
```

as.numeric(readline('Nuevo valor: '))
Pide el nuevo valor y lo convierte de texto a número con as.numeric().

datos\$Valor[i] <- nuevo
Reemplaza el valor en la posición i de la columna Valor con el nuevo número.

```
# FUNCION ELIMINAR DATO
```

```
eliminar_dato <- function(datos) {  
  fecha_eliminar <- readline("Fecha a eliminar: ")  
  nueva_base <- data.frame(  
    Fecha = character(),  
    Valor = numeric(),  
    stringsAsFactors = FALSE)  
  
  encontrado <- FALSE  
  for(i in 1:nrow(datos)) {  
    if(datos$Fecha[i] != fecha_eliminar) {  
      nueva_base <- rbind(nueva_base,datos[i,])  
    }  
  
    else {  
      encontrado <- TRUE  
    }  
  }  
  
  if(encontrado == TRUE) {  
    print("Registro eliminado")  
  }  
  
  else {  
    print("Fecha no encontrada")  
  }  
  
  return(nueva_base)  
}
```

nueva_base <- data.frame(...)

Crea un data frame vacío que servirá como copia limpia, sin el registro a eliminar.

datos\$Fecha[i] != fecha_eliminar

La condición es inversa a buscar_dato: se copian las filas cuya fecha no coincide con la que se quiere eliminar.

rbind(nueva_base, datos[i,])

Agrega cada fila conservada al nuevo data frame.

```
# MENU PRINCIPAL
programa <- function() {
  archivo <- readline("Ingrese el nombre del archivo CSV: ")
  base_datos <- cargar_datos(archivo)
  print(paste("Datos cargados:", nrow(base_datos)))
  opcion <- 0

  while(opcion != 6) {
    cat("\n===== \n")
    cat("REGISTROS:", nrow(base_datos), "\n")

    cat("===== \n")
    cat("1. Ver estadísticas\n")
    cat("2. Ver gráfica\n")
    cat("3. Buscar dato\n")
    cat("4. Editar dato\n")
    cat("5. Eliminar dato\n")
    cat("6. Salir\n")

    opcion <- as.numeric(readline("Seleccione opción: "))
  }
}
```

paste() se usa para juntar texto con otros valores (números, variables, resultados) y formar un mensaje.

cat() permite usar \n para saltos de línea y concatenar texto libremente.

Se muestra en cada iteración del menú para que el usuario vea cuántos registros quedan en la base de datos.

Lee la opción del usuario (como texto) y la convierte a número para compararla con los if/else.

```
# OPCION 1
if(opcion == 1) {
  resultados <- calcular_estadisticas(base_datos)

  cat("\n--- RESULTADOS ---\n")
  cat("Media:",round(as.numeric(resultados[1]),2),"\n")
  cat("Mediana:",round(as.numeric(resultados[2]),2),"\n")
  cat("Moda:",resultados[3],"\n")
}

# OPCION 2
else if(opcion == 2) {
  hacer_grafica(base_datos)
}

# OPCION 3
else if(opcion == 3) {
  buscar_dato(base_datos)
}

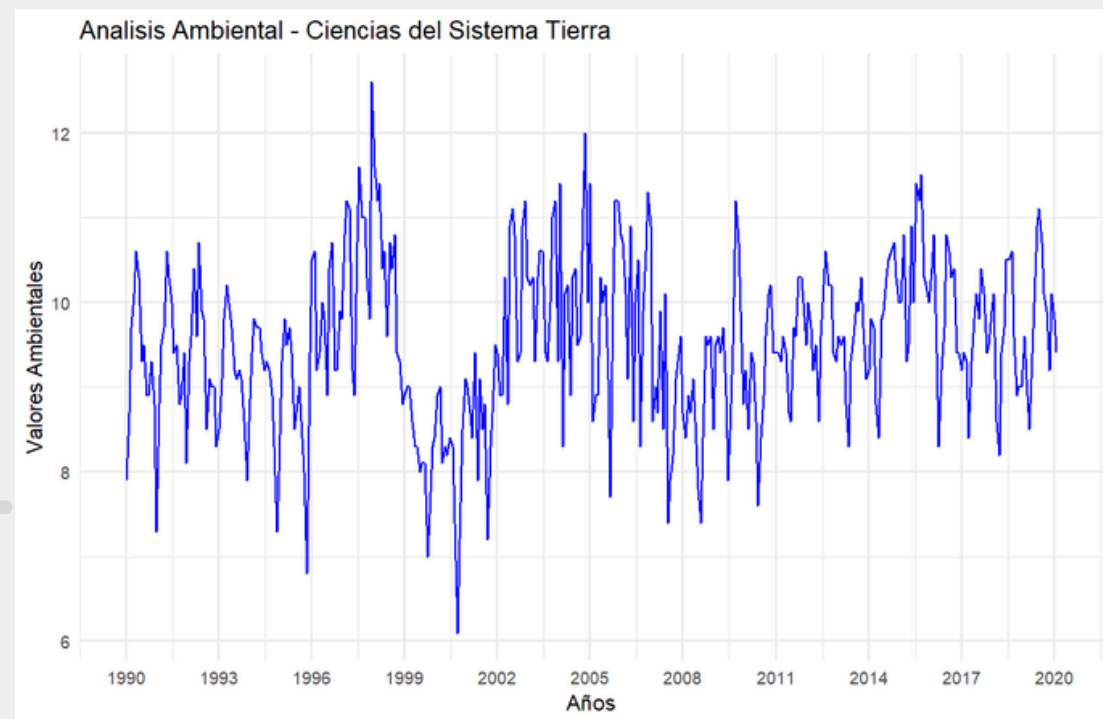
# OPCION 4
else if(opcion == 4) {
  base_datos <- editar_dato(base_datos)
}

# OPCION 5
else if(opcion == 5) {
  base_datos <- eliminar_dato(base_datos)
}
```

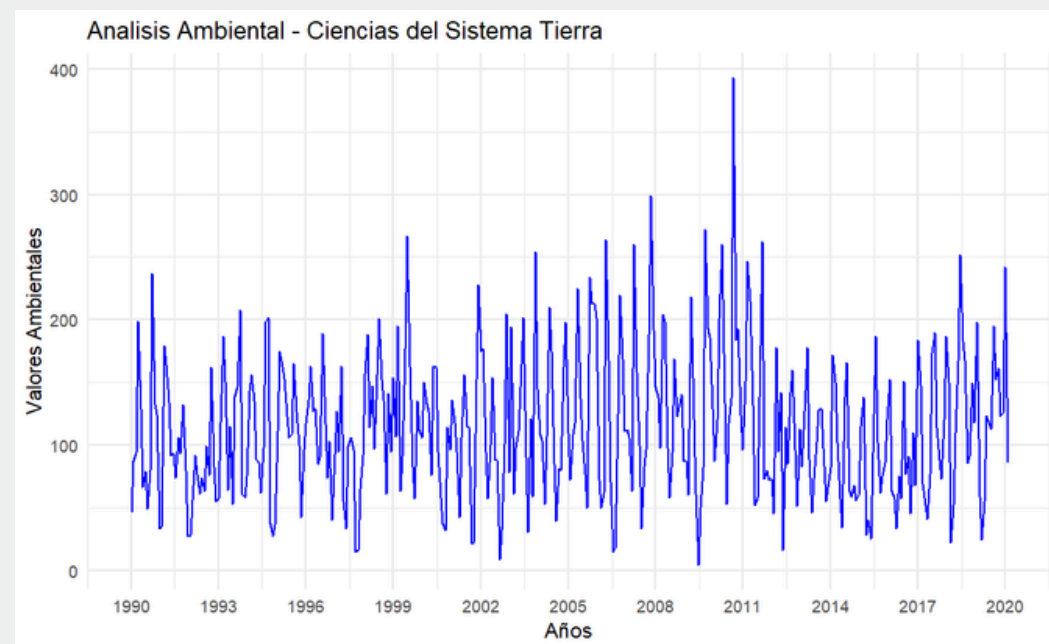
Convierte el primer elemento del vector a número y lo redondea a 2 decimales para mostrar en consola.

```
# OPCION 6
else if(opcion == 6) {
  print("Cerrando sistema...")
}
}

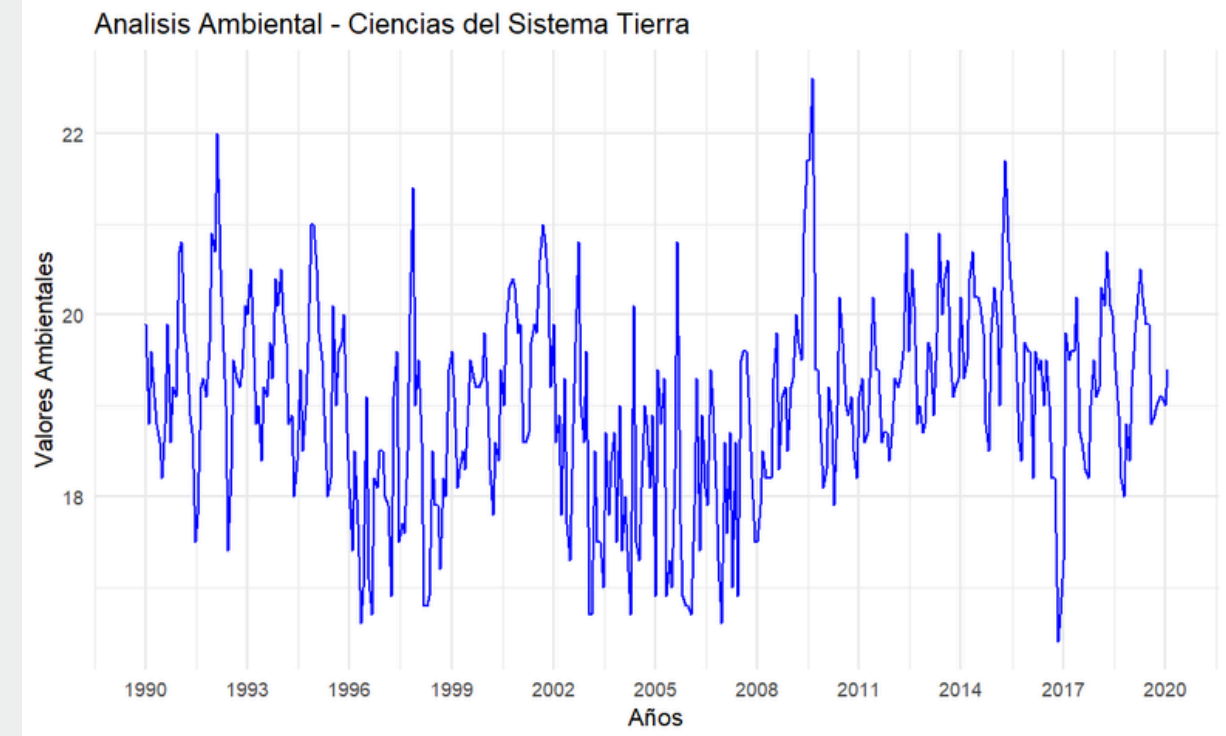
# INICIAR PROGRAMA
programa()
```



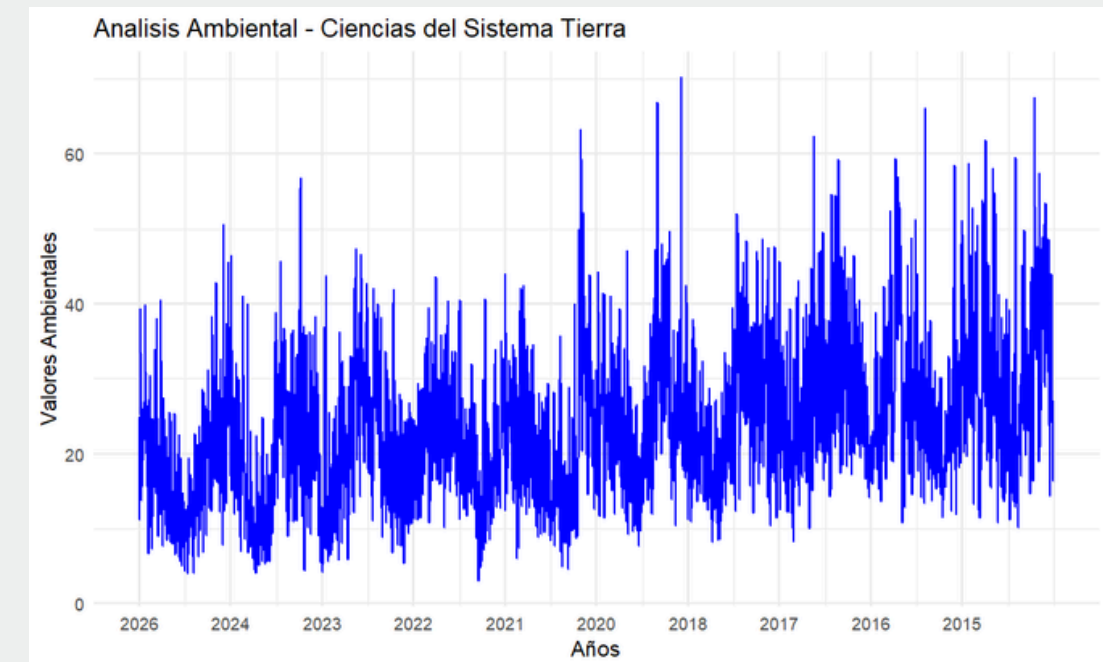
--- RESULTADOS ---
Media: 9.5
Mediana: 9.45
Moda: 9.4



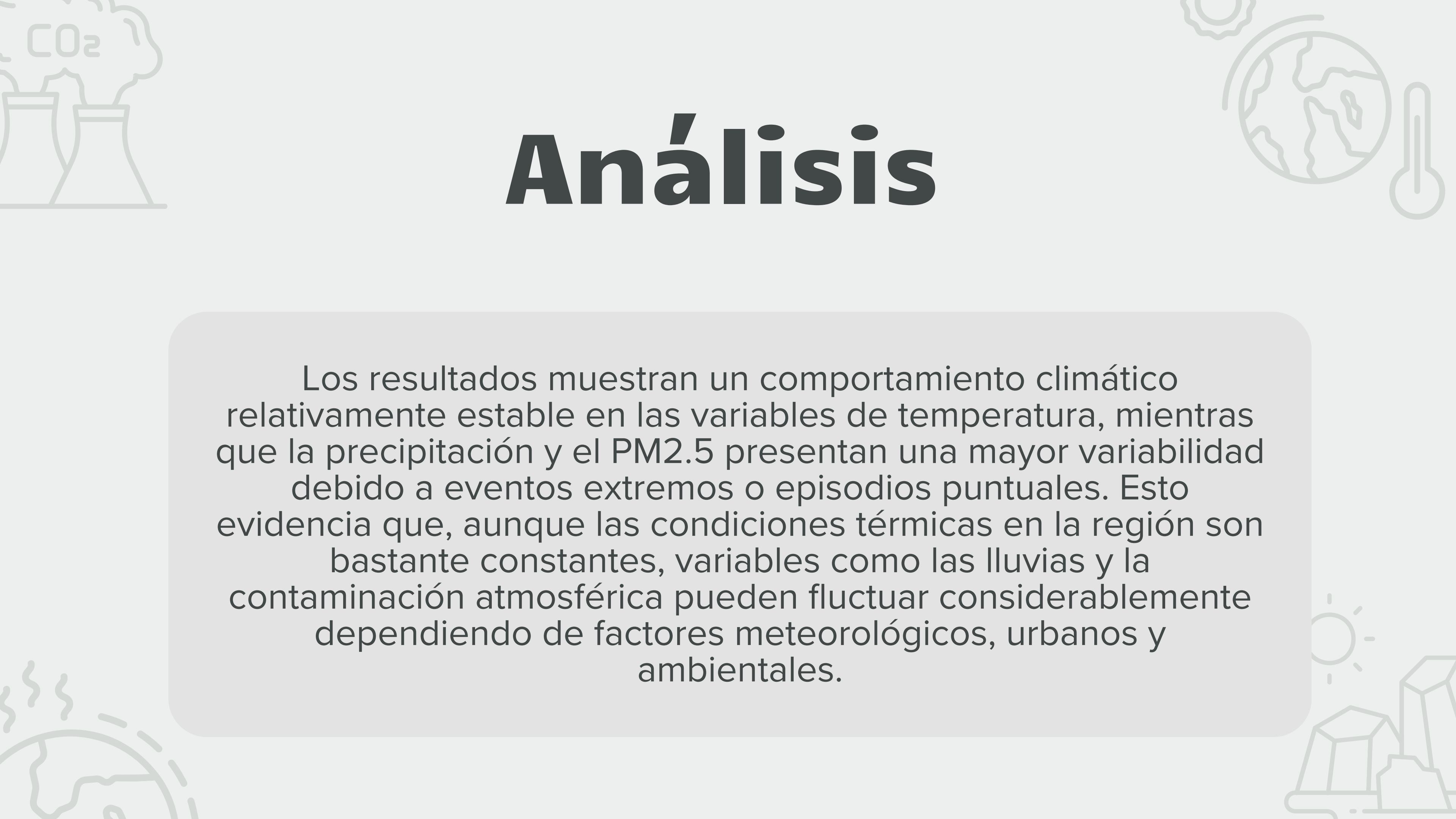
--- RESULTADOS ---
Media: 114.27
Mediana: 106
Moda: 57



--- RESULTADOS ---
Media: 19
Mediana: 19
Moda: 19

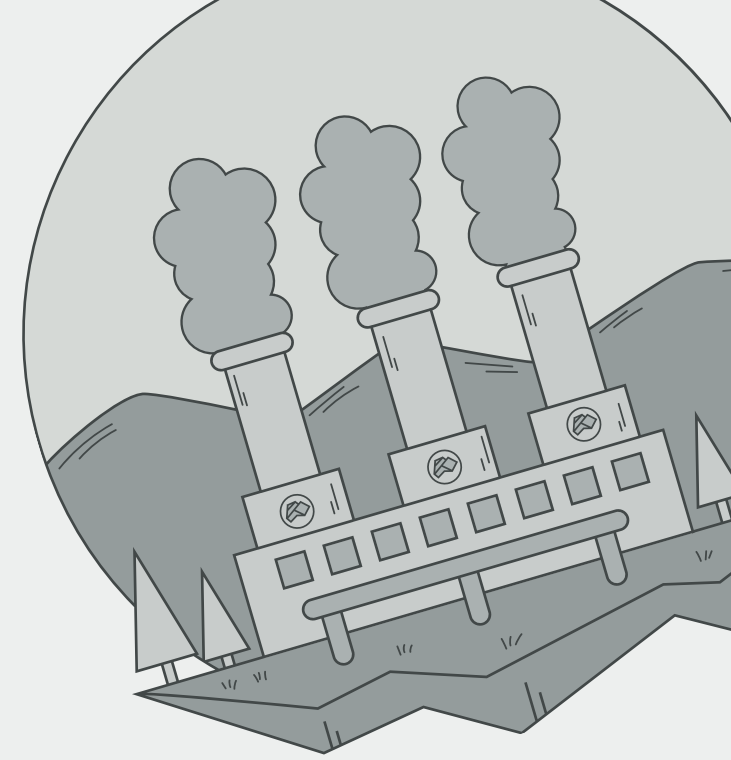


--- RESULTADOS ---
Media: 23.86
Mediana: 22.73
Moda: 19.58333333



Análisis

Los resultados muestran un comportamiento climático relativamente estable en las variables de temperatura, mientras que la precipitación y el PM2.5 presentan una mayor variabilidad debido a eventos extremos o episodios puntuales. Esto evidencia que, aunque las condiciones térmicas en la región son bastante constantes, variables como las lluvias y la contaminación atmosférica pueden fluctuar considerablemente dependiendo de factores meteorológicos, urbanos y ambientales.



Gracias

